Chapter 3. Making an Intelligent Application Agentic with MCP Clients

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. There is a GitHub repository in progress with code examples.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at *jleonard@oreilly.com*.

When working with the Model Context Protocol, you will typically be working with (or building!) clients or servers. On some projects, you may be able to build both. To really understand the protocol and use it to its full potential on your projects, you will need to be conversant with all components of the MCP architec-

First, we will dig in to host applications: what are they, what do they do, and what kinds of applications can benefit from incorporating MCP? Then, we will examine the client itself. This is what makes the host application a host: it hosts the client(s) which in turn enable the host application to communicate with MCP servers.

There is one catch, though: a single client can only talk to a single server. If you want to access multiple servers, you will have to launch multiple client instances or have a single client that only makes short, temporary connections to servers. We will examine the pros and cons of each approach, and which use cases each approach best serves.

Then, you will see a very simple host application and the client that it hosts. We will go line-by-line to help you understand the structure of a client and how it provides functionality to the host application. Finally, you will learn about best practices when it comes to building a client into a host application, ensuring that your application remains secure, reliable, and responsive.

The Host Application

Your application can be anything that hosts the client(s) that will manage connections to MCP servers, from a simple chatbot script that takes some user input and sends it to an LLM for a response to a fully-featured IDE like Cursor or Windsurf.

Thanks to the modularity of MCP, there are very few restrictions on what your host application does in order to support MCP. It only needs to communicate with an LLM and host one or more MCP clients. If you plan to use tools, then the LLM your host application uses should also support tool calling.

In the example below, you will see a barebones host application. All it does in its current state is take user input in a constant loop and pass it on to an LLM. You can find this script and all other code for this chapter in `ch3` in the book's Github repo.

Example: A Simple Host Application

```
import os

from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()

LLM_API_KEY = os.environ["LLM_API_KEY"]
```

```
anthropic_client = Anthropic(api_key=LLM_API_KEY)

print("Welcome to your AI Assistant. Type 'goodbye' to quit.")

def main():
    while True:
        prompt = input("You: ")
        if prompt.lower() == "goodbye":
            print("AI Assistant: Goodbye!")
            break
        message = anthropic_client.messages.create(
            max_tokens=4096,
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
            model="claude-sonnet-4-0",
        )
        for response in message.content:
            print(f"Assistant: {response.text}")

if __name__ == "__main__":
    main()
```

Here we have the beginnings of an MCP host: a very simple script that takes some user input and sends it to an LLM (in this case, Anthropic's Claude 3.5) and prints a response. Let's go through it line by line.

In lines 1-4, we import a few packages: `os` for loading environment variables, `anthropic` for the `Anthropic` client class, and `dotenv` for temporarily loading key-value pairs from a file into environment variables. This allows you to store sensitive information like API keys outside of the code in a file conventionally called `.env` in a `KEY=VALUE` format.

WARNING

Do not commit your .env files to version control, as this can expose your API keys to the public, leading to consequences like unauthorized use and high costs for pay-as-you-go API access. If you accidentally do this, immediately log in to your key provider and de-authorize the compromised key or keys.

In lines 6-9, the application calls `load_dotenv()` which does the work of loading your keys into environment variables, then gets the loaded key by accessing the `os.environ` dictionary. That key is then used to instantiate the Anthropic client.

After providing a welcome message to the user in line 11, the code enters an infinite loop. Line 14 requests a prompt from the user, while lines 15-17 check the prompt for the exit phrase. If it is found, the application prints a goodbye message to the user and exists.

Lines 18-28 handles communication with the model. This is a pattern you'll see over and over again when making LLM calls from code. From the Anthropic client, you access the `.messages.create()` function, and provide it with parameters like `max_tokens` to control the upper limit of tokens to generate in response to the user's query, `system` to set the system prompt, shaping how the model responds to your user, `messages` which is a list of dictionaries of prompts with a specified role (notice that we set the `role` to "user" and the `content` to the prompt from the user), and, finally the `model` specifies which model should be used.

The `messages` interface within the larger Anthropic API is very powerful, and the `create()` function has a number of other parameters that you can experiment with to tune the kinds of responses that you get from the model. You can read more at the official Anthropic documentation.

The final lines, 29-30, loop through the responses generated by the model, printing the text content of any messages received by the Anthropic client.

Next, let's shift from the Anthropic model client to the MCP client that our application will host.

The Client

When you're building support for MCP into your applications, the client is one of the most important components that you will be building and working with. Clients allow for communication between MCP servers and your application, providing the interface between the server, your application, and the large language model you're using.

Because clients act as the interface between your application and MCP servers, it is up to them to decide which server features they will support. As of this writing, MCP servers can provide to a host application:

- resources, which represent data such as text files, log files, and more
- prompts
- tools, which are executable code that an agent or model can run
- sampling, a way for a server to request chat completions from the host application's model,
- images
- context, which provides built-in MCP capabilities to tools

You can learn more about each of these in Chapter 4: The Server.

In addition to this, clients can also support **roots**, which define boundaries (such as a specific filesystem location) within which connecting servers should operate. Roots are not strictly enforced, so it is up to the server to respect them and up to the client user to review potential servers before using them.

As of this writing, there are two main transport mechanisms supported by MCP: Standard Input/Output (stdio) and Streamable HTTP. We will cover how to support each official mechanism in this chapter, and in Chapter 5 you will learn more about the transport layer itself, how official transport implementations work, and how to implement your own transport.

NOTE

Anthropic has recently released a beta version of MCP connector, which promises to allow users to access remote MCP servers directly via the Anthropic SDK's Messages API. While it seems like this could reduce the need for custom clients, as of this writing it only supports tools and remote MCP servers which, as you will see in the chapter on MCP servers, could represent a security risk. It also ties the user to Anthropic models like Claude, which may not fit your particular use case.

Basic Client Design

In MCP, a client handles all communication and connections to a server. Because client-server connections are one-to-one, it is typically best to build a client class. A client needs to at least do the following things:

- Connect to a server
- Discover the server's resources
- Make those resources available to an LLM

Beyond these basic operations, it is often helpful to implement the following features:

- Authentication
- Resource filtering
- Model agnosticism

For the rest of this chapter, you will learn how to build each of these features and what they enable for your applications. Let's start by sketching out an interface for a client class. If you are following along in the GitHub repository, the code for this section is in `client.py`, with all host application code in `agent.py`. We will continue with this structure throughout the rest of the chapter.

```
...
class MCPClient:
    def __init__(self) -> None:
        pass

    async def connect(self) -> None:
        """
        Connect to the server set in the constructor.
        """
        pass

    async def get_available_tools(self) -> list[Any]:
```

```
        """
        Retrieve tools that the server has made available.
        """
        pass

    async def use_tool(self, tool_name: str, tool_args: list |
        """
        Given a tool name and optionally a list of argumnents,
        tool
        """
        pass

    async def disconnect(self) -> None:
        """
        Clean up any resources
        """
        pass
```

In this class, you created the signatures for 3 methods, excluding the constructor:

- `connect()` - this will initialize your connection to the server. The implementation will look different based on what you choose for your transport layer
- `get_available_tools()` - this will use the client's server connection to retrieve the tools that the server makes available
- `use_tool()` - this function will call a tool given its name with any arguments provided by the caller

Another good name for your client could be `MCPServer`, as is done in the official Python examples, since from the perspective of the host application the instantiated object would represent an MCP server. This can be confusing, so for this example I decided to call it `MCPClient`. In your projects, choose what makes the most sense for you and your users.

Notice that this skeleton started with support for tools. Tools are one of the major MCP primitives, and arguably the most popular use case for MCP. Tools can help give agents their agency, augment the knowledge of LLMs, and much more, so we're starting with implementing tool support and will move on to show how to support all of the different resources an MCP server can provide.

[!NOTE] The three MCP primitives are **tools**, **prompts**, and **resources**.

Initializing the Client and Connecting to a Server

Before writing any code, let's think about *how* we want to connect to a server. This means thinking about which transport we want to use. In MCP, a **transport** is an implementation of the protocol's transport layer and manages how messages are sent between the client and server.

As of this writing, MCP has two major built-in transport implementations: stdio and streamable HTTP. The stdio transport is suitable for use cases where you expect servers will be run alongside the host application, because it uses standard input and output streams to carry communication between the client and server. This is the most common transport, as it is simple to write for and because most commercially available applications that host MCP clients expect the user to install and run servers themselves.

NOTE

The Python MCP SDK also includes a websocket transport and an HTTP Server-Sent Events (SSE) transport, which is being superseded by Streamable HTTP. Because of this, we will just cover connecting with Streamable HTTP, but the patterns established here along with Anthropic's backwards compatibility guide should give you enough to support both remote transports as a client developer. All of these transports will be explored in more detail in chapter 5.

The built-in Streamable HTTP transport, on the other hand, is best suited for applications where the MCP servers they consume can't be expected to be run alongside their host applications. This is most apparent in platform development and in hosted tools.

When developing a platform or framework for building and deploying downstream products, you can't guarantee that MCP servers will be deployed to the same machine as the application. If they're not, then a stdio client will never be able to talk to a server. Some organizations are experimenting with hosting MCP servers on their own network, exposing them to the public. If you wish to use those, you'll want to build Streamable HTTP support into your client.

WARNING

Warning: Any remote transports, including Streamable HTTP, need to be properly secured to prevent security problems. This includes authenticating the connection and validating origin headers.

First, let's look at the simplest case: connecting via stdio. In this section, you will build out the constructor and `connect()` methods for the `MCPClient` class. After that, you will do the same to build a Streamable HTTP client, which can support connecting to remote MCP servers.

After that, we will look at an example of a client supporting all three methods and how to choose the appropriate one.

Connecting with stdio

When you connect to an MCP server with stdio, the client will launch the server as a subprocess and then read messages from its standard input and send messages via its standard output. Luckily for us, the MCP Python SDK includes all the structures we will need to construct our client and connect via stdio.

First, let's set up the Client's constructor:

```
# client.py
from contextlib import AsyncExitStack
from typing import Any

from mcp import ClientSession

class MCPClient:
    def __init__(self, name: str, command: str, server_args: li
        self.name = name
        self.command = command
        self.server_args = server_args
        self.env_vars = env_vars
        self._session: ClientSession = None
        self._exit_stack: AsyncExitStack = AsyncExitStack()
        self._connected: bool = False
    ...
```

In this constructor, you set up several important properties that will enable you to connect to and interact with an MCP server. The first, `name` just gives a human-readable name to the instantiated client. This is useful for logging, especially if you are maintaining connections to multiple servers, because it allows you to easily identify the specific origin of any emitted logs. It is not, however, required for an MCP client.

The next 3 parameters are used for setting up the server. `command` identifies the executable to be run to initiate the server. Most commonly, servers will run with either `python` or `node` (or `npx` depending on your local installation). It is common enough that for most use cases, your client could define an enum to ensure that a supported command is being used. `server_args` is a list of all the command-line arguments that are to be passed to the `command` executable. This will include, at the very least, the path to the server file. If the file takes command-line arguments, let's say `--port 8000 --verbose`, your `server_args` list will look like `["server.py", "--port", "8080"]`. Next is a dictionary of environment variables in `env_vars`. These work like typical environment variables, and you can even unpack your system's environment variables into this dictionary with `{**os.env}`. This isn't recommended, however, as it could expose sensitive information to your server.

We also include some "private" connection management properties, such as `_session`, which will hold the client session, `_exit_stack` which manages the asynchronous connection contexts (more on that later), and `_connected` which will prevent us from connecting to a server when we're already connected.

The next step is to implement the `connect()` and `disconnect()` methods. Here, you'll use the Python MCP SDK's built-in connection management objects to make and hold the server connection.

```
# client.py
```

```
from contextlib import AsyncExitStack
from typing import Any

from mcp import ClientSession
from mcp.client.stdio import StdioServerParameters, stdio_clien

class MCPClient:
    ...
    async def connect(self) -> None:
        """
        Connect to the server set in the constructor.
        """
        if self._connected:
            raise RuntimeError("Client is already connected")

        server_parameters = StdioServerParameters(
            command=self.command,
            args=self.server_args,
            env=self.env_vars if self.env_vars else None
        )

        # Connect to stdio server, starting subprocess
        stdio_connection = await self._exit_stack.enter_async_c
        self.read, self.write = stdio_connection

        # Start MCP client session
        self._session = await self._exit_stack.enter_async_cont

        # Initialize session
        await self._session.initialize()
        self._connected = True

    async def disconnect(self) -> None:
        """
        Clean up any resources
        """
        if self._exit_stack:
            await self._exit_stack.aclose()
            self._connected = False
            self._session = None
```

This may look complex, especially if you don't have any experience with writing asynchronous Python. Let's break down what we're doing here to make it more digestible, starting with the `connect()` method. The very first thing this method does is check the `_connected()` property. This will prevent the user from trying to open an already-open connection. Then, we instantiate a `StdioServerParameters` object, passing in the properties we set in the constructor. The rest of the code is creating the necessary processes and connections to run and use the MCP server. We do this via `_exit_stack`, which holds an instance of `AsyncExitStack`.

[!NOTE] `AsyncExitStack` allows you to manually nest asynchronous context managers without using the more common `async with` context manager syntax. This is an ideal choice because you can dynamically add more asynchronous context managers to the stack and can safely release all the resources on the stack in order with just one call. The latter feature is extremely useful because we can determine when to unwind the stack, rather than having to rely on the code leaving the scope of each context manager. This is done by calling the `enter_async_context()` function.

First, we open a connection to the stdio server, which starts the subprocess on which the MCP session will run. We pass in an instance of the `stdio_client` that is instantiated with the `server_params` built just before it. Then, we unpack the results of that call into `self.read` and `self.write` which represent the session's read and write streams, respectively. The next step is to start the MCP client session, again by calling `enter_async_context()`, but this time with an instance of `ClientSession` with the read and write streams set to `self.read` and `self.write`. This is stored in `self._session`, which we then initialize with `initialize()`. This method handles initiating the connection to the server, advertising the client's capabilities to the server, checking the protocol version supported by the server, and telling the server that the client has been successfully initialized. Finally, we set `self._connected` to `True`.

In sum, when writing the connection code for a stdio server, you:

1. create an instance of `StdioServerParams`
2. start the stdio server subprocess in an asynchronous context
3. start the MCP client session in another (nested) asynchronous context and

store it in `self._session`
4. initialize the session itself

In this example, we also created the `disconnect()` method, which just calls `self._exit_stack.aclose()` to close all server connections in order, then sets `self._connected` to `False` and `self._session` to `None`. With this, you can cleanly connect and disconnect to MCP servers using the stdio transport at will. While you can't do anything with the servers just yet, you've completed the most complex part. Give yourself a pat on the back. Now we will do the same thing, but in support of the Streamable HTTP transport.

Connecting with Streamable HTTP

Third-party example Streamable HTTP is the main transport designed for use with remote MCP servers. Originally, HTTP Server-Sent Events (SSE) were the standard for connecting to remote servers, but a number of production issues with them made their continued support untenable. Those issues, and the architecture of both transports, will be covered in Chapter 5.

When you connect to a remote server with Streamable HTTP, you will be connected via a single endpoint defined by the server, and then responses can be either immediate standard HTTP responses (triggered by sending a POST to the server and not requiring an always-open connection) or streaming SSE responses (triggered by sending an empty GET to the server), the latter being optional and dependent on the server's implementation.

NOTE

“Wait,"” you might be saying, “I thought Streamable HTTP replaces HTTP+SSE, how is this different?"” We will go into the upgrades in Chapter 5, but the major difference with this is that getting an SSE response is optional - both by the client that has to request a streaming response, and by the server that can choose whether or not to support it.

To support Streamable HTTP, we will again focus on building the `MCPClient`'s constructor, `connect()`, and `disconnect()` methods. Let's start with the constructor:

```
# client.py
from contextlib import AsyncExitStack
from typing import Callable

from mcp import ClientSession

class MCPClient:
    def __init__(self, name: str, server_url: str) -> None:
        self.name = name
        self.server_url = server_url
        self._session: ClientSession = None
        self.exit_stack = AsyncExitStack()
        self._connected: bool = False
        self._get_session_id: Callable[[], str] = None
```

This is eerily similar to the constructor for the stdio client. Because we're not starting a process locally, we don't need `command` or `server_args`, but we do need a location for the server, which `server_url` provides. The rest of the properties are the same in the stdio client.

Connecting and disconnecting from the server is also pretty similar to how you do it for the stdio client:

```
# client.py
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

class MCPClient:
    ...
    async def connect(self, headers: dict | None = None) -> Nor
        if self._connected:
            raise RuntimeError("Client is already connected")

        # Connect to Streamable HTTP server
        streamable_connection = await self._exit_stack.enter_as
        self.read, self.write, self._get_session_id = streamabl

        # Start MCP client session
```

```
        self._session = await self._exit_stack.enter_async_cont

        # Initialize session
        await self._session.initialize()
        self._connected = True

    async def disconnect(self) -> None:
        """
        Clean up any resources
        """
        if self._exit_stack:
            await self._exit_stack.aclose()
            self._connected = False
            self._session = None
```

Hopefully this code looks very familiar - it's almost exactly the same as the `connect()` and `disconnect()` code for the stdio client. What we do differently here is take an optional `headers` parameter and pass that with the `server_url` property to `streamablehttp_client()`, which along with the `read_stream` and `write_stream` returns a callback for retrieving a session ID from the server if it provides one. If the server supports it, this can be used for resuming broken sessions. We also don't instantiate a `StdioServerParams` object, because the `streamablehttp_client` function doesn't take a special parameters object, instead taking the URL and headers directly as parameters. Other parameters that would be of interest for a client developer include:

- `timeout` - how long in seconds until HTTP operations time out, of type `datetime.timedelta`. Defaults to 30 seconds.
- `sse_read_timeout` - how long in seconds to wait for an additional event before timing out, of type `datetime.timedelta`. Defaults to 5 minutes.
- `auth` - this handles authentication and is of type `httpx.auth`. You will learn how to authenticate sessions with servers that support it in a later section.

While you are building out your client class, let's take a look at one strategy for integrating it into a host chatbot application. We are going to use the stripped-down chatbot introduced at the beginning of this chapter to demonstrate how the MCP client fits in with a wider LLM-powered application. The only additions here will be to instantiate an stdio MCP client and open a connection to a made-up MCP server that provides calculator tools like `add_two_numbers`, `subtract_two_numbers`, `multiply_two_numbers`, and `divide_two_numbers`.

```
import asyncio
from pathlib import Path
...
mcp_client = MCPClient(
    name="calculator_server_connection",
    command="uv",
    server_args=[
        "--directory",
        str(Path(__file__).parent.resolve()),
        "run",
        "calculator_server.py",
    ],
)

print("Welcome to your AI Assistant. Type 'goodbye' to quit.")

def main():
    await mcp_client.connect()
    while True:
        prompt = input("You: ")
        if prompt.lower() == "goodbye":
            print("AI Assistant: Goodbye!")
            break
        message = anthropic_client.messages.create(
            max_tokens=4096,
            messages=[
                {
                    "role": "user",
                    "content": prompt,
                }
            ],
            model="claude-sonnet-4-0",
        )
        for response in message.content:
            print(f"Assistant: {response.text}")
    await mcp_client.disconnect()
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

All that we did here was instantiate the client and then call `connect()` to open a connection to the server. Then, after the chat loop is exited, we call `disconnect()` to close the connection. How would you change this for a Streamable HTTP server?

NOTE

When instantiating the client, our command is `uv` instead of `python`. `uv` is useful here because it can be run from anywhere and makes managing virtual environments and dependencies simple. While you can change the command to `python` or `python3`, be sure to check out `uv`.

Now let's look at how to make use of everything an MCP server can provide to the client.

Interacting with MCP Server Capabilities

MCP servers can provide a wide variety of capabilities, including the MCP primitives *tools*, *resources*, and *prompts*. As a client developer, you will interact with them via the instantiated `ClientSession` object, and generally use the same two-step workflow for each of these: discovery and use. For each of these capabilities, discovery happens via a `<primitive>/list` request, which in the Python SDK is wrapped by the `list_<primitive>s()` methods. These calls, if the server provides the respective capability, will return a list of the available tools, resources, or prompts. Using the resulting capabilities requires a `<primitive>/<verb>` call, where the verb depends on the primitive being called. In the Python SDK, these calls are wrapped with methods of the form `<verb>_<primitive>()` such as `call_tool()` for `tool/call`.

NOTE

The protocol also allows notification messages to be sent by the server (as well as by the client), then it is up to the receiver to handle the notifications as it wishes. For Streamable HTTP connections, these notifications will be sent like any other message as defined by the transport, but will be sent as a `JSONRPCNotification`, and implementation of the `JSONRPCMessage` class.

While the available MCP SDKs wrap the various request calls in methods that your client can call directly, it's often a good idea to wrap them in your own object methods so that you can add logging, retries, method parameters and any other customizations to your calls to the server, like passing any of the `list_<primitive>()` methods the optional `cursor` parameter for pagination.

Tools

Tools are the most common resource MCP servers provide, and with good reason: agentic workflows often require tools to augment the abilities of a base LLM. Tools are simply a deterministic function that an LLM can decide to call if needed. Naturally, the function can do anything you can think of doing in code, from calculating numbers to finding the weather at a given location. MCP servers allow clients to discover and use these tools.

To discover tools using the Python SDK, your client would just call `self.session.list_tools()` within the client, which returns a `response` object with a `tools` property. This holds a list of any tools made available by the server connected to the client, where tools have the properties `name`, `description`, `inputSchema`, `annotations`, and `model_config`. You can see the current structure of the Tool class in the MCP Python SDK's types module.

Calling `list_tools()` directly can be fine for basic use cases, but if you're building a client that will be used for different servers with different capabilities and possibly in different applications, it can be advantageous to write your own function that has some guardrails, logging, and anything else your users might need abstracted away. For example, you could implement the following function:

```
# client.py
import logging
...
logger = logging.getLogger(__name__)
```

```
class MCPClient:
    ...
    async def get_available_tools(self) -> list[dict[str, Any]]
        if not self._connected:
            raise RuntimeError("Client not connected to a serve

        tools_result = await self._session.list_tools()
        if not tools_result.tools:
            logger.warning("No tools found on server")
        available_tools = [
            {
                "name": tool.name,
                "description": tool.description,
                "input_schema": tool.inputSchema,
            }
            for tool in tools_result.tools
        ]
        return available_tools
...
# agent.py
async def main():
    await mcp_client.connect()
    available_tools = await mcp_client.get_available_tools()
    print(f"Available tools: {", ".join([tool['name'] for tool
    while True:
        ...
```

This method is a very simple wrapper around the `Session` object's `list_tools()` method which itself wraps the `tools/list` request. While simple, the example shows how you can implement some simple guardrails like the server connection status check, as well as more detailed logging like the empty tool result check. The method also converts the MCP-native tool object list into a list of dictionaries with the keys "name", "description", and "input_schema". This ensures compatibility with the Anthropic messages API. You can extend this pattern to add retries, translate MCP's tool format to another format compatible with another LLM, and more. Another helpful pattern that you will see later in this chapter is converting MCP's `Tool` objects into your own objects where you can customize their structure, implement translation functions to support other LLMs, and more. In the `main()` function, we added a list comprehension to build a list of the available tool names and printed them to ensure that our code is working.

Now that your client has tools, what do you do with them? Use them, of course! Like in the `get_available_tools()` example, we will wrap the `Session` object's `call_tool()` function and add just a bit of logging to it and handle the different available response types. `call_tool()` returns a `CallToolResult`, which is a list of any of the SDK's content types:

- `TextContent` - The result of the tool is any kind of text that you would return to user, and it's stored in the `text` property.
- `ImageContent` - If the result of the tool is an image, it will be returned as an `ImageContent` object, with the image data itself stored as a base64-encoded string in the `data` property
- `AudioContent` - Some tools return audio content. Like `ImageContent`, the data is base64-encoded and lives in the `data` property
- `EmbeddedResource` - Tools can return embedded resources as part of their response, typically for extra context or data caching. The resource data lives in the `resource` property, which can be either `TextResourceContents` (representing the text content of, say, a configuration file resource) or `BlobResourceContents` (representing the binary data of the resource as a base64-encoded string)

One key thing to keep in mind is that tool use requests return lists of content, and in some cases, such as with an `EmbeddedResource` response being used as additional context, that should be expected and handled as such. In the example below, you will wrap `call_tool()` and return string representations of each of the valid tool content response types.

WARNING

For some use cases, handling the response types in the client itself may not be desirable. When building a client, consider your user: will they want or need control over how to handle the different content types?

```
# client.py
```

```
from typing import Any
from mcp.types import TextResourceContents

class MCPClient:
    ...
    async def use_tool(self, tool_name: str, arguments: dict[st
        if not self._connected:
            raise RuntimeError("Client not connected to a serve

        tool_call_result = await self._session.call_tool(name=t
        logger.debug(f"Calling tool {tool_name} with arguments

        results = []
        if tool_call_result.content:
            for content in tool_call_result.content:
                match content.type:
                    case "text":
                        results.append(content.text)
                    case "image" | "audio":
                        results.append(content.data)
                    case "resource":
                        if isinstance(content.resource, TextRes
                            results.append(content.resource.tex
                        else:
                            results.append(content.resource.blo
        else:
            logger.warning(f"No content in tool call result for
        return results
```

Here, we built a `use_tool()` method that, like the `list_tools()` method, first checks that the client is connected to a server session and raises an exception if it isn't. Then we make the `call_tool()` call, passing in the tool's name and arguments. On the next line, we added a debug statement so we know what tool is being called. If you want the user to decide how to handle the different content types, you could return the results immediately at this point. But for this example, we check that the `content` list is not empty, then iterate through each item in it. There, we check the item's type and add the proper string representation to our results list. If `content` is empty, we show the user a warning, and then finally return the results list. Just like with `list_tools()`, wrapping `call_tool()` gives us much more control over what happens during the tool call.

WARNING

When building agents that support MCP, keep a close eye on tool-calling and overall performance as you add tools to the agent. Performance tends to drop dramatically as more tools are added, typically due to an increased difficulty in selecting the correct tool for a prompt. This can be due to ambiguity or overlap in tool descriptions as well as descriptions and interfaces having too much information in aggregate for the LLM to process accurately.

If you're building your client to serve a specific application, like the simplified chat application introduced in the beginning of this chapter, you will also have to use these calls. Many LLM APIs that support tool-calling have an optional `tools` parameter, making sending the tools available to a model simple. Other models require building a list of tool names, descriptions, and schemas into your system prompt, but for this example we are focusing on Anthropic's Claude API, which does accept a `tools` parameter. This will be added to any user messages to the model, and then the resulting responses will be analyzed by the host application for any tool usage. If it's found, the application will formulate a `tool_result` message and add it to a messages list that gets sent back to the model to generate the final result for the user.

the tools made available by the MCP server, and allow it to call them.

```
# agent.py
...
mcp_client = MCPClient(
    name="calculator_server_connection",
    command="uv",
    server_args=[
        "--directory",
        str(Path(__file__).parent.resolve()),
        "run",
        "calculator_server.py",
    ],
)

print("Welcome to your AI Assistant. Type 'goodbye' to quit.")
```

```python
async def main():
    """Main async function to run the assistant."""
    try:
        await mcp_client.connect()
        available_tools = await mcp_client.get_available_tools(
        print(
            f"Available tools: {", ".join([tool['name'] for too
        )

        while True:
            prompt = input("You: ")
            if prompt.lower() == "goodbye":
                print("AI Assistant: Goodbye!")
                break

            # Build conversation starting with user message
            conversation_messages = [{"role": "user", "content"

            # Tool use loop - continue until we get a final tex
            while True:
                # Get LLM response
                current_response = anthropic_client.messages.cr
                    max_tokens=4096,
                    messages=conversation_messages,
                    model="claude-sonnet-4-0",
                    tools=available_tools,
                    tool_choice={"type": "auto"},
                )

                # Add assistant message to conversation
                conversation_messages.append(
                    {"role": "assistant", "content": current_re
                )

                # Check if we need to use tools
                if current_response.stop_reason == "tool_use":
                    # Extract tool use blocks
                    tool_use_blocks = [
                        block
                        for block in current_response.content
                        if block.type == "tool_use"
                    ]

                    # Execute all tools and collect results
                    tool_results = []
                    for tool_use in tool_use_blocks:
                        print(f"Using tool: {tool_use.name}")
                        tool_result = await mcp_client.use_tool
                            tool_name=tool_use.name, arguments=
                        )
                        tool_results.append(
                            {
                                "type": "tool_result",
                                "tool_use_id": tool_use.id,
                                "content": "\n".join(tool_resul
                            }
                        )

                    # Add tool results to conversation
                    conversation_messages.append(
                        {"role": "user", "content": tool_result
                    )

                    continue

                else:
                    # No tools needed, extract final text respo
                    text_blocks = [
                        content.text
                        for content in current_response.content
                        if hasattr(content, "text") and content
                    ]

                    if text_blocks:
                        print(f"Assistant: {text_blocks[0]}")
                    else:
                        print("Assistant: [No text response ava

                    break
    finally:
        await mcp_client.disconnect()


if __name__ == "__main__":
```

```
    asyncio.run(main())
```

This code has a number of changes to handle getting and using any required tools. First, during the initialization phase, we get all available tools from the server we’re connected to with `get_available_tools()`. Then, we transform the list of `Tool` objects in to a list of plain dictionaries to properly feed it back to the model. This is done at the top of the module in order to only pull the list of tools once. If you expect the server or servers you connect to to change the tools they provide while connected, you will want to either get a fresh list of available tools for every user input to the model or have your client listen to and handle a `list_changed` notification from the server.

Next, we’ve updated the code where we create the user message to the model to pass in the list of tools to the `tools` parameter, as well as setting `tool_choice` to a dictionary with the key `"type"` set to ` “auto”``. You don’t have to do this, since auto is the default setting and allows the model to decide which tool or tools to use to answer the user’s query. You can also set it to `"any"``, where it will use any (but at least one) tool that is available to it, or a tool name and type `"tool"` to force the model to use a specific tool, or `"none"` to prevent the model from using any available tools.

After this, we inspect the result and initialize a potential tool use message that will be sent back to the model. If the message response has a `stop_reason` of `tool_use`, then we know the model is waiting for a response with the results of the tool or tools that it is requesting. We create tool use message blocks from the response contents, and then execute any tools in a loop. From the results, we build a dictionary with the key `type` set to `"tool_use"`, `tool_use_id` set to the ID from the model’s tool use message response, and `content` to the result of the tool call. All of those are then added to the `conversation_messages` list. Once we’ve made all of the tool calls, the results and the original user question are sent beck to the LLM for evaluation. Because there are potentially multiple tool calls that could be made before returning to the user (such as asking a model what 5 * 3 + 7 is), we wrap all of this in a while loop.

If we don’t have any tool call responses in the tool use message, either no tools were necessary or that we’ve come to the end of the tool use phase. Either way, we extract the text blocks from the latest response and print them if available. Finally, the whole thing is wrapped in a try/finally block to ensure that the connection is disconnected from the proper scope when the user quits the application.

In this section, you learned about MCP server-provided tools, how to discover them on a server, how to use them, and how to use them in the context of a chat application. In the next section, you will learn about resources and how you can use those in your application like you just did with tools.

Resources

As you learned in Chapter 2, MCP resources can play many roles, some of which have yet to be fully explored by the community members themselves, like as a cache to optimize prompt size. Basically, resources give a host application and the LLM it interacts with access to data served by the MCP server. This can be database records, images, text files, and more. Resources also have several actions you can take on them:

`resources/list` *or* *`list_resources()`*

> requests a list of available resources from the server

`resources/templates/list` *or* *`list_resource_templates()`*

> requests a list of resource URI templates from the server. These allow you to construct a resource’s URI dynamically based on other information you might have. In the Python SDK, these strings look like f-strings, with variable sections enclosed in curly braces.

`resources/read` *or* *`read_resource()`*

> accesses the file provided by the server, sending the data in text or blob format to the LLM.

`resources/subscribe` *or* *`subscribe_resource()`*

> subscribes the client to updates to to the resource URI you specify in the call.

`resources/unsubscribe` or `unsubscribe_resource()`

unsubscribes the client from updates to the resource specified by resource URI in the parameters.

NOTE

See the tupac project by Tim Kellogg for a minimalist client implementation that utilizes MCP resources as a cache for efficient prompting.

First, we will implement wrappers for the two list methods. We will continue the pattern that we established in the last section for `get_available_tools()`: a light wrapper with a few basic guardrails and logging that can be used as a base for more complex workflows.

```
# client.py
from mcp.types import Resource, ResourceTemplate

class MCPClient:
    ...
    async def get_available_resources(self) -> list[Resource]:
        if not self._connected:
            raise RuntimeError("Client not connected to a serve

        resources_result = await self._session.list_resources()
        if not resources_result.resources:
            logger.warning("No resources found on server")
        return resources_result.resources

    async def get_available_resource_templates(self) -> list[Re
        if not self._connected:
            raise RuntimeError("Client not connected to a serve

        resource_templates_result = await self._session.list_re
        if not resource_templates_result.resources:
            logger.warning("No resource templates found on serv
        return resource_templates_result.resources
```

These two methods are almost identical to each other and to the `get_available_tools()` method. Again, each function checks for a connection before making any server requests, then calls the appropriate list method, checks for a result, and then returns it, including a warning if the result is empty. Like the `list_tools()` method, both of these list methods take an optional cursor parameter for paginating results. We will see this pattern again for prompts in the next section, so think about how you could expand the functionality of the list method wrappers and how you could also make the code more concise.

NOTE

Resources and resource templates have almost the exact same set of properties: `uri`, `name`, `description`, `mimeType`, `size`, `annotations`, and `model_config`. For resource templates, instead of a `uri`, they have a `uriTemplate` property.

Next, we'll implement a wrapper of the `read_resource()` session method, and following the pattern the previous example established, we'll call it `use_resource()`. This code will take a resource URI (or a filled in URI from a resource template), read the resource and return it to the caller either as a text string or a base64-encoded data string.

```
# client.py
from mcp.types import BlobResourceContents, TextResourceContent

class MCPClient:
    ...
    async def get_resource(self, uri: str) -> list[BlobResource
        if not self._connected:
            raise RuntimeError("Client not connected to a serve
        resource_read_result = await self._session.read_resourc

        if not resource_read_result.contents:
            logger.warning(f"No content read for resource URI {
        return resource_read_result.contents
```

This method requires little extra: just our now-standard connection guardrail and empty result warning. Similarly to `get_available_tools()`, instead of transforming the MCP `BlobResourceContents` and `TextResourceContents` classes, we return them as-is to the user, since they provide valuable information about the type of string being returned and this keeps with the minimal processing philosophy of this set of examples.

The MCP protocol also allows clients to subscribe to resource change notifications, however as of this writing, notification handling is only partially implemented in the Python SDK and there is no provided way to route notifications to handler functions or register handlers. This does seem to be available in the Typescript SDK, however. There is a message handler callback in the ClientSession class, but it handles all incoming messages, including requests, responses, and notifications.

So far, you learned how to get a list of available resources and resource templates and, when armed with a resource URI, how to read that resource. But how would you use them in a host application? Remember that resources primarily exist to provide additional context to user queries, allowing your application to do things like adding a Bash script to a user query about their script. To see this in action, let's use our minimal host chatbot application example from the beginning of the chapter again, but this time we will look for the key "context:" in the user prompt and use that as context to the model request. When the application starts and any time the user issues the command "refresh," the application will get the current list of resources from the server and then display them. When the application finds "context:" in the user prompt, it will use what comes after to select a specific resource and add it as context to the original user question.

If you're following along with the code in the Github repository, you'll notice that the code for this next example has been split into two files: `client.py` and `agent.py`. This is to help reduce the complexity of the code, and to make it more readable. The following code example will begin with `agent.py`, which now contains the `Agent` class. This class will hold the previous `main()` function as `run()`, and will also hold a few utility functions for handling resources. These are `extract_resource_name()` which extracts the resource name from the user prompt, `display_resources()` which prints the available resource names and descriptions to the user, and `refresh_resources()` which refreshes the available resources and returns them as a dictionary.

```python
# agent.py
...
class Agent:
    def __init__(self, mcp_client: MCPClient, anthropic_client:
        self.mcp_client = mcp_client
        self.anthropic_client = anthropic_client
        self.available_resources = {}

    async def _select_resources(self, prompt: str) -> list[str]
        """Use LLM to intelligently select relevant resources."
        if not self.available_resources:
            return []

        resource_descriptions = {
            name: resource.description or f"Resource: {name}"
            for name, resource in self.available_resources.item
        }

        selection_prompt = f"""
Given this user question: "{prompt}"

And these available resources:
{json.dumps(resource_descriptions, indent=2)}

Which resources (if any) would be helpful to answer the user's
Return a JSON array of resource names, or an empty array if no
Only include resources that are directly relevant.

Example: ["math-constants"] or []
"""

        try:
            response = self.anthropic_client.messages.create(
                max_tokens=200,
                messages=[{"role": "user", "content": selection
                model="claude-sonnet-4-0",
            )

            response_text = response.content[0].text.strip()
            # Extract JSON from response (handle case where LLM
            if "[" in response_text and "]" in response_text:
                start = response_text.find("[")
                end = response_text.rfind("]") + 1
```

```
                json_part = response_text[start:end]
                selected_resources = json.loads(json_part)
                return [r for r in selected_resources if r in s

        except Exception as e:
            logger.warning(f"Failed to select resources with LL

        return []

    async def _load_selected_resources(
        self, resource_names: list[str]
    ) -> list[dict[str, Any]]:
        """Load the specified resources."""
        context_messages = []

        for resource_name in resource_names:
            if resource_name in self.available_resources:
                print(f"LLM selected resource: {resource_name}"
                try:
                    resource = self.available_resources[resourc
                    resource_contents = await self.mcp_client.g
                        uri=resource.uri
                    )
                    for content in resource_contents:
                        if isinstance(content, TextResourceCont
                            context_messages.append(
                                {
                                    "type": "text",
                                    "text": f"[Resource: {resou
                                }
                            )
                        elif content.mimeType in [
                            "image/jpeg",
                            "image/png",
                            "image/gif",
                            "image/webp",
                        ]:  # b64-encoded image
                            context_messages.append(
                                {
                                    "type": "image",
                                    "source": {
                                        "type": "base64",
                                        "media_type": content.m
                                        "data": content.blob,
                                    },
                                }
                            )
                        else:
                            print(
                                f"WARNING: Unable to process mi
                            )
                except Exception as e:
                    print(f"Error loading resource {resource_na

        return context_messages

    async def _refresh_resources(self) -> None:
        available_resources = await self.mcp_client.get_availab
        self.available_resources = {
            resource.name: resource for resource in available_r
        }

    async def run(self):
        try:
            print(
                "Welcome to your AI Assistant. Type 'goodbye' t
            )
            await self.mcp_client.connect()
            available_tools = await self.mcp_client.get_availab
            await self._refresh_resources()

            while True:
                prompt = input("You: ")

                if prompt.lower() == "goodbye":
                    print("AI Assistant: Goodbye!")
                    break

                if prompt.lower() == "refresh":
                    await self._refresh_resources()
                    continue

                selected_resource_names = await self._select_re
                context_messages = await self._load_selected_re
```

```python
                    selected_resource_names
                )

                # Build conversation with initial user message
                user_content = [{"type": "text", "text": prompt
                if context_messages:
                    user_content.extend(context_messages)

                conversation_messages = [{"role": "user", "cont

                # Tool use loop - continue until we get a final
                while True:
                    # Get LLM response
                    current_response = anthropic_client.message
                        max_tokens=4096,
                        messages=conversation_messages,
                        model="claude-sonnet-4-0",
                        tools=available_tools,
                        tool_choice={"type": "auto"},
                    )

                    # Add assistant message to conversation
                    conversation_messages.append(
                        {"role": "assistant", "content": curren
                    )

                    # Check if we need to use tools
                    if current_response.stop_reason == "tool_us
                        # Extract tool use blocks
                        tool_use_blocks = [
                            block
                            for block in current_response.conte
                            if block.type == "tool_use"
                        ]

                        print(f"Executing {len(tool_use_blocks)

                        # Execute all tools and collect results
                        tool_results = []
                        for tool_use in tool_use_blocks:
                            print(f"Using tool: {tool_use.name}
                            tool_result = await self.mcp_client
                                tool_name=tool_use.name, argume
                            )
                            tool_results.append(
                                {
                                    "type": "tool_result",
                                    "tool_use_id": tool_use.id,
                                    "content": "\n".join(tool_r
                                }
                            )

                        # Add tool results to conversation
                        conversation_messages.append(
                            {"role": "user", "content": tool_re
                        )

                        # Continue loop to get next LLM respons
                        continue

                    else:
                        # No tools needed, extract final text r
                        text_blocks = [
                            content.text
                            for content in current_response.con
                            if hasattr(content, "text") and con
                        ]

                        if text_blocks:
                            print(f"Assistant: {text_blocks[0]}
                        else:
                            print("Assistant: [No text response

                        # Exit the tool use loop
                        break
        finally:
            await self.mcp_client.disconnect()


if __name__ == "__main__":
    mcp_client = MCPClient(
        name="calculator_server_connection",
        command="uv",
        server_args=[
```

```
            "--directory",
            str(Path(__file__).parent.parent.resolve()),
            "run",
            "calculator_server.py",
        ],
    )
    agent = Agent(mcp_client, anthropic_client)
    asyncio.run(agent.run())
```

This specitific integration started just like the integration you did in the Tools section: connect to the MCP server via the client, discover available primitives, like resources in this case, and load those primitives into memory. We added two new utility functions to the agent:

- `_select_resources()` : This creates a name: description dictionary of available resources, and then uses the LLM to select any relevant resources given the user's prompt.
- `_load_selected_resources()` : This takes a list of resource names from `_select_resources()` and for valid resource names, it uses the client to get the resource contents. If those contents are text, we create a content block with the text. If it is a blob, we check that the `mimeType` matches any of the Anthropic model API-supported image types and if so, construct a content block with that data and add that to a final list of messages to be send back to the caller.
- `_refresh_resources()` : this refreshes the available resources and stores them in the `available_resources` dictionary on the Agent object.

In the `run()` function, we updated the welcome message to let the user know that a message with only the word `refresh` will refresh the resource list. The resource list is then filled with the results of the `refresh()` method. Then once we're prompted by the user, we, we get the LLM to select any resources pertinent to the user prompt with `_select_resources()` . Then we call `_load_selected_resources()` to add any named resources to the list of `context_messages` . Context messages provide additional context to the LLM, and we bundle them with the user prompt in a list. This list of user prompt and selected resources is then sent to the LLM, and the model will be able to use the resource contents to answer the user's prompt if needed.

NOTE

When doing things like adding context to a user prompt, you should not create a separate user message like you do with the user prompt. Instead, you convert the user message to its full form: a nested dictionary with a `type` key and a `content` key, which holds a list of dictionaries and add any context messages to that list.

There are several improvements you can make to this example. To stretch your knowledge, try to implement support for listing and using resource templates along with normal resources. Then, try to support multiple context files instead of just one, either with regular text parsing or even another LLM call. You can do some more cleanup: currently, our application does 1 extra potentially expensive LLM csll for each user input in order to select the most relevant resources for the user's query. How could you make this more efficient?

NOTE

As you might have picked up on, many of these improvements are typical software engineering best practices and not really specific to generative AI. While AI engineering is becoming a discipline unto itself, it is still software engineering at its core and thus the practices and tools you have as a software engineer will also serve you well building for AI.

Prompts

Prompts are the last of the three main primitives provided by MCP servers. These are designed to be controlled by the application user, who should be able to pick out which prompt or prompts they want to use, similar to the interface you saw in the host application example in the previous section. Each prompt definition has a unique identifier `name` , an optional human-readable `description` , and optional `arguments` , which is a list of dictionaries with a `name` , optional human-readable `description` and an optional `required` boolean indicating if the argument is required or not. It is then up to the client implementation or host application using the prompt to inject values for each of the arguments. These are called **dynamic prompts**.

The Python SDK simplifies handling prompts for us, with the `Session` class's `list_prompts()` method returning a list of `Prompt` objects, which have the prompt definition properties listed in the previous paragraph. When you get a specific prompt with `get_prompt()`, you provide the prompt's name and arguments and get back a list of `PromptMessage` objects, which are ready-to-use messages that can be converted to a plain dictionary and sent directly to the LLM. Take a look at the host integration example in the last section, paying close attention to the structure of the messages that are sent to the LLM. They are dictionaries with `role` and `content` keys, and those are the two required properties of a `PromptMessage` object. The content itself can be `TextContent`, `ImageContent`, `AudioContent`, or an`EmbeddedResource`. Embedded resources are just what you think they are: resources that are embedded in a message content block, just like our example host application built in the previous section.

Let's continue the pattern from the previous two sections and wrap these two methods in our own client:

```
# client.py
from mcp.types import Prompt

class MCPClient:
    ...
    async def get_available_prompts(self) -> list[Prompt]:
        if not self._connected:
            raise RuntimeError("Client not connected to a serve

        prompt_result = await self._session.list_prompts()
        if not prompt_result.prompts:
            logger.warning("No prompts found on server")
        return prompt_result.prompts
```

There isn't too much to explain about this pattern that you haven't seen before. We set up a connection check, get the list of prompts, and ensure that the result list isn't empty before returning it to the user. The `get_prompt()` wrapper will be very familiar as well. We will get the prompt name and arguments from the user, load the prompt from the server with the provided arguments, and return the MCP `PromptMessage` object back to the application.

```
# client.py
from mcp.types import PromptMessage

class MCPClient:
    ...
    async def load_prompt(self, name: str, arguments: dict[str,
        if not self._connected:
            raise RuntimeError("Client not connected to a serve
        prompt_load_result = await self._session.get_prompt(nam

        if not prompt_load_result.messages:
            logger.warning(f"No prompt found for prompt {name}"
        else:
            logger.warning(f"Loaded prompt {name} with descript
        return prompt_load_result.messages
```

This is again similar to how we retrieved resources in the last section. We have our standard guardrails, we pass the `name` and `arguments` unchanged to the session's `get_prompt()` method, and add some logging.

NOTE

As with resources and tools, in the protocol there is support for notifications when the list of available prompts has changed. Unfortunately, there is also no user-facing API for handling and listening for these notifications from the server, although clients can subscribe to them. If you want to use them, you will have to handle routing to a callback on your own, but there are a few examples in the SDK's source code you can follow, such as how sampling callbacks are routed in `Session._received_request()`. Source code. You could also create a new function that handles all messages and pass that to the client constructor in the `message_handler` parameter. This would have to handle all messages, but would allow you to build notification handling logic as well.

Like the other primitives, there are as many ways to integrate prompts into your application as you can imagine, and then some. To show off how you can use the available prompt functionality in your application, we will use the same base application as the last two examples so that the usage of the server and prompt primitives is clear and not obscured by too much application logic. We will also use a similar user interaction pattern to the one we used in the Resources section. In this pattern, the application gets a list of available primitives (in this case, prompts), displays them to the user, and allows the user to select which one to use within their input. That prompt will be sent as-is by itself to the LLM and the application will handle and display the response.

with prompts.

```
# agent.py
class Agent:
    def __init__(self, mcp_client: MCPClient, anthropic_client:
        self.mcp_client = mcp_client
        self.anthropic_client = anthropic_client
        self.available_resources = {}
        self.available_prompts = {}

    async def _select_resources(self, user_query: str) -> list[
        """Use LLM to intelligently select relevant resources."
        ...

    async def _select_prompts(self, user_query: str) -> list[di
        """Use LLM to intelligently select relevant prompts."""
        if not self.available_prompts:
            return []

        prompts = [
            prompt.model_dump_json() for prompt in self.availab
        ]

        selection_prompt = f"""
Given this user question: "{user_query}"

And these available prompt templates:
{json.dumps(prompts, indent=2)}

Which prompts (if any) would provide helpful instructions or gu
Return a JSON array of prompt objects which have a name (string
keys are the named parameter name and value is the argument val
are needed. Only include prompts that are directly relevant.

Example: [{{"name": "calculation-helper", "arguments": {{"opera
 {{"name": "step-by-step-math", "arguments": {{}}}}] or []
"""

        try:
            response = self.anthropic_client.messages.create(
                max_tokens=200,
                messages=[{"role": "user", "content": selection
                model="claude-sonnet-4-0",
            )

            response_text = response.content[0].text.strip()
            if "[" in response_text and "]" in response_text:
                start = response_text.find("[")
                end = response_text.rfind("]") + 1
                json_part = response_text[start:end]
                selected_prompts = json.loads(json_part)
                return [
                    p for p in selected_prompts if p["name"] in
                ]

        except Exception as e:
            logger.warning(f"Failed to select prompts with LLM:

        return []

    async def _load_selected_resources(
        self, resource_names: list[str]
    ) -> list[dict[str, Any]]:
        ...

    async def _load_selected_prompts(self, prompts: list[dict[s
        """Load the specified prompts as system instructions.""
        system_instructions = []

        for prompt in prompts:
            if prompt["name"] in self.available_prompts:
                print(f"Using prompt: {prompt['name']}")
                try:
                    prompt_content = await self.mcp_client.load
                        name=prompt["name"], arguments=prompt["
```

```
                    )

                    # Extract the prompt text
                    prompt_text = ""
                    for message in prompt_content:
                        if hasattr(message.content, "text"):
                            prompt_text += message.content.text
                        elif isinstance(message.content, str):
                            prompt_text += message.content + "\

                    if prompt_text.strip():
                        system_instructions.append(
                            f"[Prompt: {prompt['name']}]\n{prom
                        )

                except Exception as e:
                    print(f"Error loading prompt {prompt['name'

        return "\n\n".join(system_instructions)

    async def _refresh(self) -> None:
        available_resources = await self.mcp_client.get_availab
        self.available_resources = {
            resource.name: resource for resource in available_r
        }
        available_prompts = await self.mcp_client.get_available
        self.available_prompts = {prompt.name: prompt for promp

    async def run(self):
        try:
            print(
                "Welcome to your AI Assistant. Type 'goodbye' t
            )
            await self.mcp_client.connect()
            available_tools = await self.mcp_client.get_availab
            await self._refresh()

            print(
                f"Loaded {len(self.available_resources)} resour
            )

            while True:
                prompt = input("You: ")

                if prompt.lower() == "goodbye":
                    print("AI Assistant: Goodbye!")
                    break

                if prompt.lower() == "refresh":
                    await self._refresh()
                    continue

                # Select relevant resources and prompts
                selected_resource_names = await self._select_re
                selected_prompt_names = await self._select_prom

                # Load relevant resources and prompts
                context_messages = await self._load_selected_re
                    selected_resource_names
                )
                system_instructions = await self._load_selected
                    selected_prompt_names
                )

                # Build conversation with initial user message
                user_content = [{"type": "text", "text": prompt
                if context_messages:
                    user_content.extend(context_messages)

                conversation_messages = [{"role": "user", "cont

                # Tool use loop - continue until we get a final
                while True:
                    create_message_args = {
                        "max_tokens": 4096,
                        "messages": conversation_messages,
                        "model": "claude-sonnet-4-0",
                        "tools": available_tools,
                        "tool_choice": {"type": "auto"},
                    }

                    if system_instructions:
                        create_message_args["system"] = system_
```

```
                    current_response = self.anthropic_client.me
                        **create_message_args
                    )

                    # Add assistant message to conversation
                    conversation_messages.append(
                        {"role": "assistant", "content": curren
                    )

                    # Check if we need to use tools
                    if current_response.stop_reason == "tool_us
                        # Extract tool use blocks
                        tool_use_blocks = [
                            block
                            for block in current_response.conte
                            if block.type == "tool_use"
                        ]

                        # Execute all tools and collect results
                        tool_results = []
                        for tool_use in tool_use_blocks:
                            print(f"Using tool: {tool_use.name}
                            tool_result = await self.mcp_client
                                tool_name=tool_use.name, argume
                            )
                            tool_results.append(
                                {
                                    "type": "tool_result",
                                    "tool_use_id": tool_use.id,
                                    "content": "\n".join(tool_r
                                }
                            )

                        # Add tool results to conversation
                        conversation_messages.append(
                            {"role": "user", "content": tool_re
                        )

                        # Continue loop to get next LLM respons
                        continue

                    else:
                        # No tools needed, extract final text r
                        text_blocks = [
                            content.text
                            for content in current_response.con
                            if hasattr(content, "text") and con
                        ]

                        if text_blocks:
                            print(f"Assistant: {text_blocks[0]}
                        else:
                            print("Assistant: [No text response

                        # Exit the tool use loop
                        break
        finally:
            await self.mcp_client.disconnect()
...
```

This version of the code does all the things we set up in the introduction to this example. Starting with the main loops, we first call a `_refresh_()` function that calls the `get_available_prompts()` method that we built into the client, and fills the `available_prompts` attribute with a dictionary similar to the Resources example, where the keys are the prompt names and the values are the prompt objects. In the agent's loop, we call `_select_prompts()` to get the LLM to select the most relevant prompts for the user's query, just like in the Resources section, with just a few differences. First, we dump the `Prompt` object to a JSON string with `model_dump_json()`, allowing our selection prompt to have the full prompt object - including the description and arguments - to help it decide on the best prompt or prompts to use. Then we call `_load_selected_prompts()` to filter any hallucinated prompts, get any remaining prompts via the client, and then transform them into the system prompt format.

In this example, before calling the LLM, we create a dictionary of message arguments, then add the loaded prompts to those arguments (if there are any) with the `system` argument key. We then unpack these arguments in Anthropic's `create()` call, but the rest of the agent remains the same.

Table of contents collapsed